

Q1. Delete an element from the front of a LL

example $\rightarrow$ i/p $\Rightarrow$ head $\downarrow$

1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ null

i/p head
is null

O/p $\Rightarrow$

head $\downarrow$

2 $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ null

head = head.next
 $\longleftrightarrow$

head $\downarrow$

1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ null

$\downarrow$

hc
C++ (delete)

TC $\Rightarrow O(1)$
SC $\Rightarrow O(1)$

Q2. Delete an element from end of LL

example: head $\downarrow$

1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ null

O/p $\Rightarrow$ head $\downarrow$

1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ null

temp = head

while (temp.next != null) {

temp = temp.next

}

head $\rightarrow$ null
 $\longleftrightarrow$

1 head.next = null
 $\rightarrow$

temp.next = null

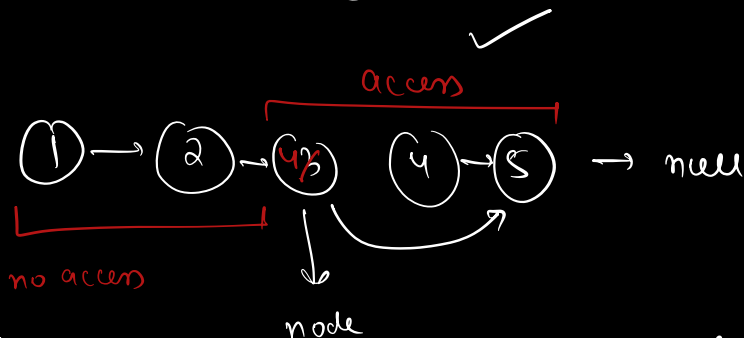
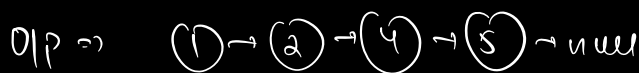
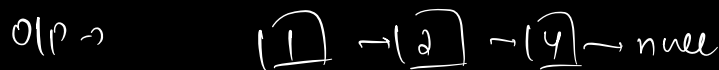
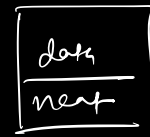
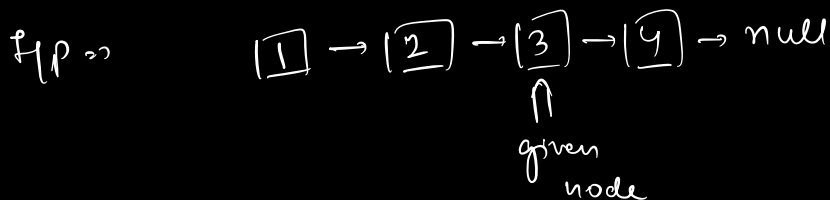
1

$$TC \Rightarrow O(N)$$

- i) No element
- ii) 1 element
- iii) 2 element
- iv) 3 element

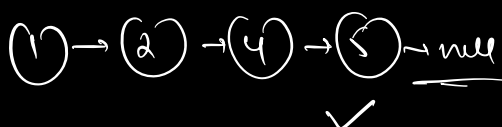
Google

Q1. Given a node of a LL, delete this node. Also the given node is not the last node of LL, should be done $O(1)$



node.data = 3

node.next.data = 4



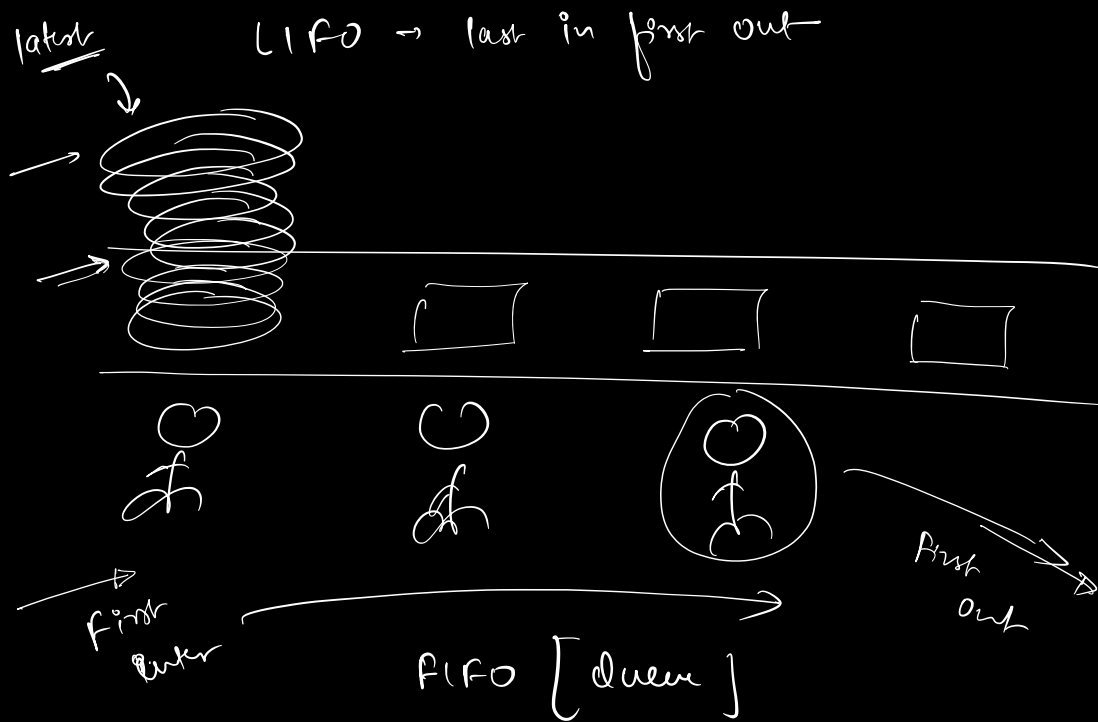
i) node.data = node.next.data

ii) node.next = node.next.next

$$TC \Rightarrow O(1)$$
$$SC \Rightarrow O(1)$$

⇒ Stacks:-

abstract data structure



⇒ Stack

func / feature ⇒

mandatory

push(data) → [inserting in stack]

pop() → [removing from stack]

peek() / top() → [returns the top most value without removing]

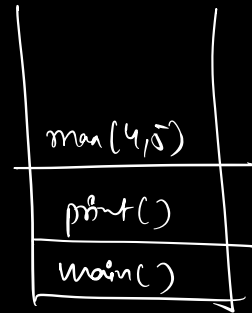
isEmpty() → size is 0 or not

size() → current size

⇒ Applications of Stack:

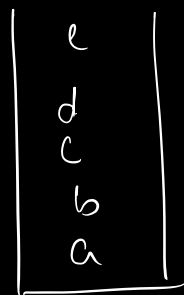
i) Call stack

```
main {  
    print(max(4, 5))  
}
```

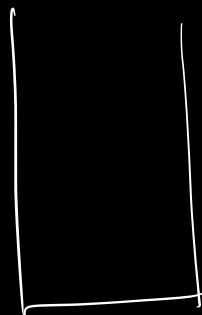


ii)

a b c d e



main

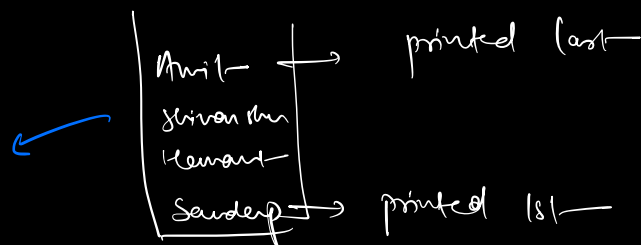


undo/redo

iii) back / forward button of a browser.

printer ⇒

Queue



Queue

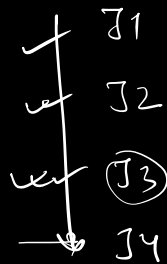
abstract data structure [FIFO]

func/feature $\Rightarrow$
mandatory $\swarrow$ [enqueue (push/add)
dequeue (pop/remove/poll)]

$\Rightarrow$ Applications for queue:

i) Kafka | RabbitMQ | Redis | ActiveMQ $\downarrow$
messaging queues [msg]

ii) Scheduling



iii) print scheduling $\Rightarrow$ Queue

✓ i) Array

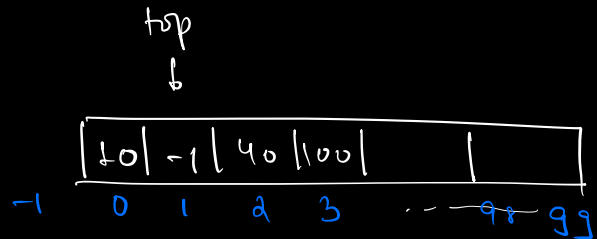
ii) Arraylist
(n.u.s)

iii) Linkedlist
✓

⇒ Impl of Stack using array

```
int arr[100]
```

```
int top = -1
```



O(1)

```
void push( data ) {  
    if ( top < 99 ) {  
        top++;  
        arr[top] = data;  
    }  
}
```

O(1)

```
int pop() {  
    if ( top > -1 ) {  
        temp = arr[top];  
        top--;  
        return temp;  
    }  
}
```

push(10)

push(20)

push(30)

pop()

push(40)

push(80)

pop()

push(100)

pop() ←

pop()

pop() push(-1)

⇒ Implementation of Stack using LL

$O(1)$

↖ add At front

↘ add At Back

$O(1)$

$O(1)$

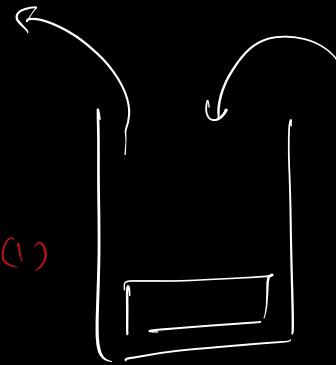
remove from front ↖

remove from Back ↘

$O(N)$

push(a) → add At front (a) → $O(1)$

pop() → remove from front() → $O(1)$



push(10)

push(40)

pop() ←

push(20)

push(80)

pop()

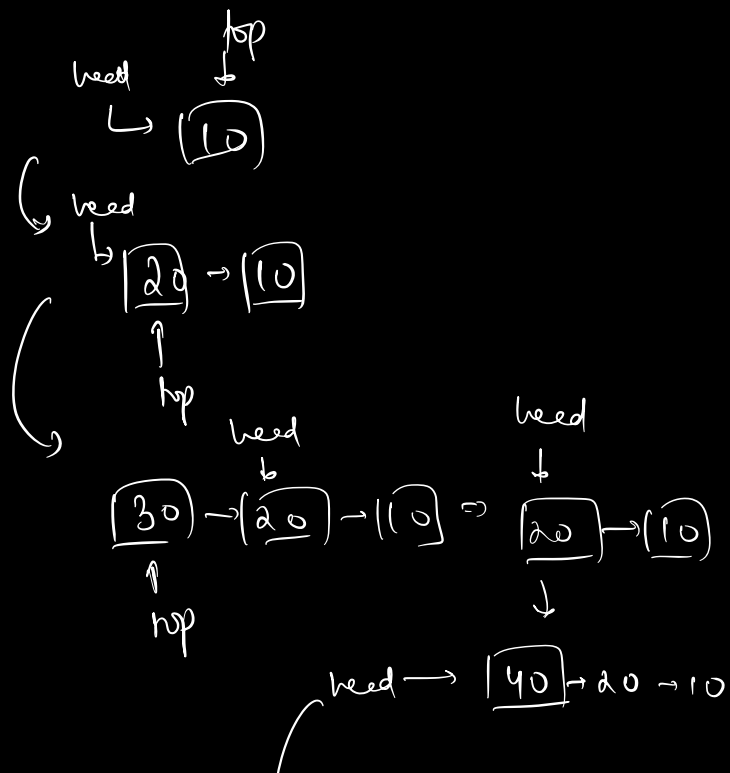
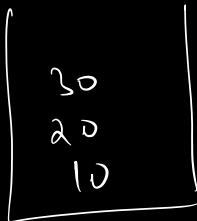
push(30)

pop()

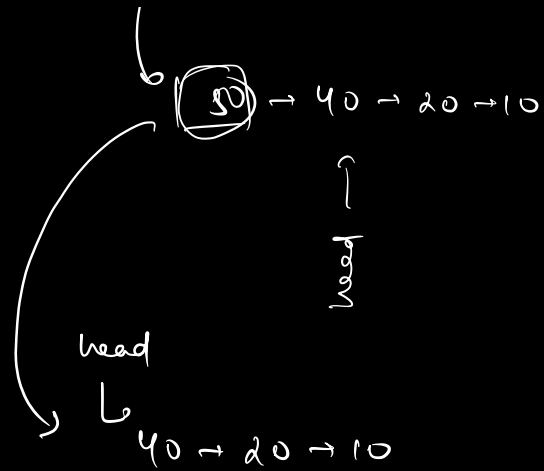
pop() push(-1)

pop()

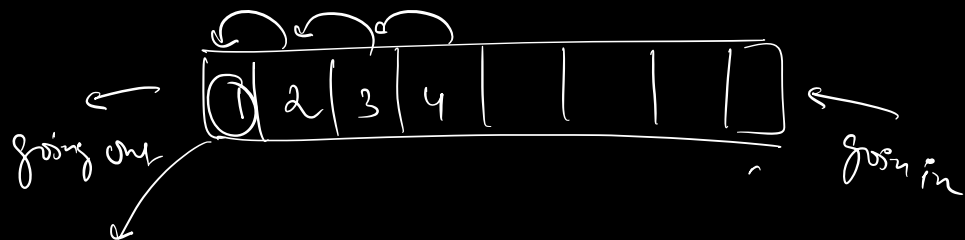
push(100)



50
40
20
10



queue using array



enq(1)

enq(2)

enq(3)

enq(4)

deq

enq() $\rightarrow O(1)$
deq() $\rightarrow O(N)$

enq(10)

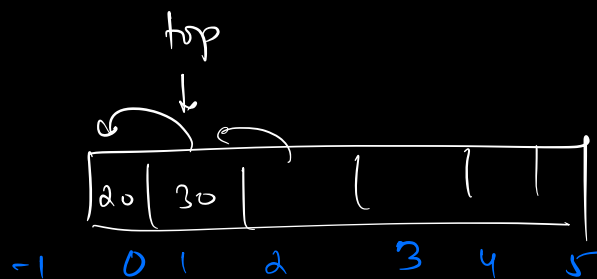
enq(20)

enq(30)

deq()

temp = 10

top--;



shift all elements to left by 1

Queue using LL

$O(1)$

addAt front

addAt Back

$O(1)$

$O(1)$

remove from front

remove from Back

$O(N)$



enq(1)

enq(2)

enq(3) ←

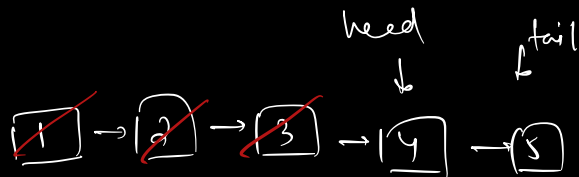
deq() → ①

enq(4) ✓

enq(5)

deq()

deq()

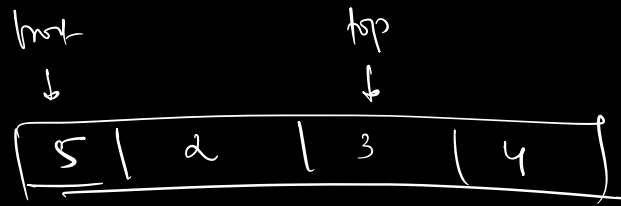


enq → addAt Back

deq → remove from front

$O(1)$

choice



max $\Rightarrow$ 4

curr $\Rightarrow$ 3

①